

Tendências de Motores de Jogos

# Semana 3

Material, Landscape, Nanite, Lumen e o primeiro Build

**Disciplina:** Tendências de Motores de Jogos (IN46A)

**Instituição:** IFMS — Campus Dourados

**Curso:** Tecnologia em Jogos Digitais

**Módulo 1 — Aprender a Ferramenta**

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

## Objetivos da Semana

### OBJETIVOS

- Compreender o Material Graph como shader nodal universal
- Modelar um terreno básico via Landscape
- Compreender Nanite e Lumen como soluções para dois problemas universais de renderização
- Compreender Packaging e gerar o primeiro build executável do Vertical Slice

## Como a semana está organizada

---



### **Encontro 1**

Material Graph + criação de material simples + Landscape (terreno)



### **Encontro 2**

Nanite + Lumen + Packaging + Checkpoint/Showcase de encerramento do Módulo 1

Tendências de Motores de Jogos

## ENCONTRO 1

# Material Graph e Landscape

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



**Como uma engine decide a cor, o brilho e o relevo de uma superfície sem código de shader escrito à mão?**

Pense em texturas, valores numéricos e operações matemáticas combinados visualmente.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

## Material Graph: shader nodal universal

- **Texture Sample** — lê os pixels de uma textura
- **Scalar Parameter** — valor numérico ajustável (ex.: rugosidade)
- **Vector Parameter** — cor ou vetor ajustável
- Tudo se conecta aos pinos finais: Base Color, Roughness, Metallic, Normal
- **Material Domain** define o uso do material (ex.: Surface)



### DICA

Todo material resolve o mesmo problema: transformar textura + parâmetros em aparência de superfície.

## Tendências de Motores de Jogos

# Da textura à superfície



IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

## Shader nodal: Unreal × Unity

### Unreal

Material Graph integrado nativamente ao pipeline desde a primeira versão, com Material Domains e blend modes no próprio grafo

### Unity

Shader Graph pleno apenas nos pipelines modernos (URP/HDRP); histórico dividido com o Built-in Render Pipeline



## Demonstração: criando o M\_Ground

O professor cria `M_Ground` em `Materials/Base`, conecta uma textura do Kenney ao Base Color via Texture Sample, e um Scalar Parameter ( `Roughness_Amount` ) ao Roughness.

**O que muda:** a superfície de pré-visualização passa de cinza genérica para uma textura reconhecível e ajustável.

### *Imagem sugerida*

*Objetivo: mostrar o Material Graph do M\_Ground já conectado.*

*Enquadramento: Material Editor centralizado no grafo de nós.*

*Elementos presentes: nó Texture Sample conectado ao Base Color; nó Scalar Parameter*

*( **Roughness\_Amount** ) conectado ao Roughness; painel de pré-visualização com a esfera de exemplo.*

*Destaque visual: contorno nas duas conexões (Base Color e Roughness) que levam ao nó principal do material.*

*Legenda sugerida: "Textura e parâmetro convergindo para as propriedades finais da superfície."*

Tendências de Motores de Jogos



# Por que não usar um Static Mesh comum como terreno?

Pense em uma superfície de centenas de metros, com variações suaves de altura.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

## Landscape: geometria de terreno em larga escala

- Heightmap define a elevação do terreno ponto a ponto
- Ferramentas de **Sculpt** (elevar, abaixar, suavizar) em tempo real
- **Paint Layers** aplicam material sobre a superfície esculpida
- Resolve um problema diferente do material: geometria, não aparência



### DICA

Material resolve aparência; Landscape resolve geometria de terreno. Nunca confundir os dois.

## Terreno: Unreal × Unity

### Unreal

Landscape — heightmap + Paint Layers, sculpt em tempo real dentro do editor

### Unity

Terrain — papel equivalente: heightmap + camadas de material

## **Demonstração: esculpindo o terreno**

O professor cria o Landscape no `Map_Exploration`, esculpe pequenas variações de elevação com a ferramenta Sculpt, suaviza com Smooth, e pinta a superfície com o `M_Ground` via Paint Layer.

**Por que:** um terreno visualmente reconhecível, sem comprometer a locomoção já testada do BP\_Player.

## Arquitetura: Material e Landscape no projeto

- `Materials/Base/M_Ground` — convenção `M_` do PROJECT\_ARCHITECTURE.md
- `Textures/` — primeira semana com assets externos ao template (Kenney)
- Landscape do `Map_Exploration` — geometria do nível de teste

### NA INDÚSTRIA

O mesmo nível de teste que já hospeda o BP\_Player agora ganha aparência e geometria próprias.

## Boas práticas

### ✓ BOAS PRÁTICAS

- Nomear parâmetros pela propriedade que representam ( `Roughness_Amount` ), nunca `Param1`
- Esculpir o terreno em pequenos incrementos, testando a locomoção a cada ajuste
- Nunca usar Static Mesh comum como substituto de terreno em larga escala



# Laboratório do Encontro 1

Cada grupo cria seu próprio `M_Ground` e esculpe o terreno do `Map_Exploration` com Landscape, aplicando o material à superfície.

## OBJETIVOS

Critério de sucesso: material simples aplicado a um terreno básico via Landscape, com o BP\_Player se movendo normalmente sobre ele, seguindo a convenção `M_` do PROJECT\_ARCHITECTURE.md.

## Fechando o Encontro 1

---

- Material Graph resolve aparência; Landscape resolve geometria de terreno
- `M_Ground` criado e aplicado ao terreno esculpido no `Map_Exploration`
- Próximo encontro: renderização moderna (Nanite, Lumen) e o primeiro build

Tendências de Motores de Jogos

## ENCONTRO 2

# Nanite, Lumen e Packaging

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



# **Como exibir geometria muito detalhada sem comprometer a performance a cada quadro?**

Pense no custo de renderizar milhões de polígonos, e na solução tradicional de LODs manuais.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

## Nanite: geometria virtualizada

- Problema universal: complexidade geométrica custa caro a cada quadro
- Solução tradicional: **LODs** manuais, trocados por distância da câmera
- Nanite exibe apenas o detalhe necessário **por pixel**, sem LODs manuais



### DICA

Nanite não aumenta o limite de polígonos — muda como a geometria é processada e exibida.

## Lumen: iluminação global dinâmica

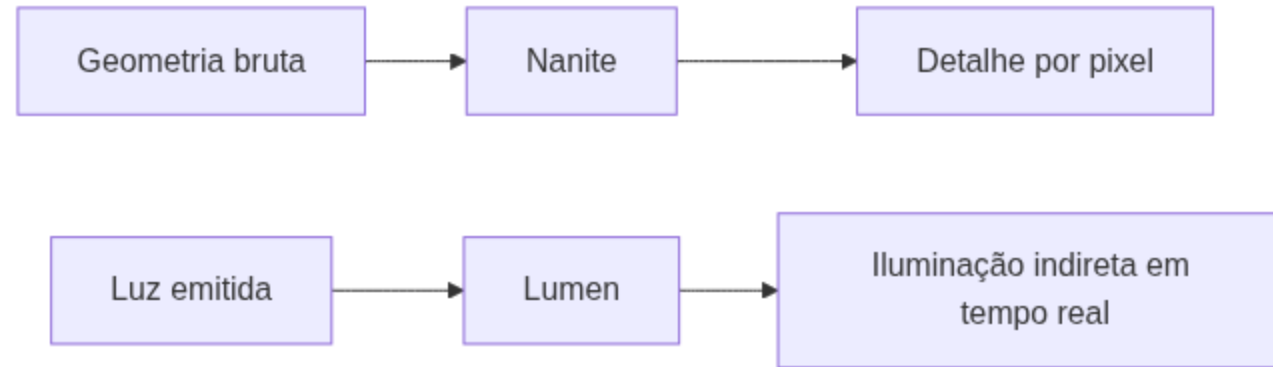
- Problema universal: simular luz que rebate entre superfícies (iluminação indireta)
- Solução tradicional: **bake de lightmaps**, desatualizado se a cena mudar
- Lumen calcula iluminação indireta **em tempo real**, sem bake prévio



### DICA

Lumen calcula a propagação da luz existente — não gera luz onde não há fonte alguma.

## Dois problemas, duas soluções



## Renderização moderna: Unreal × Unity

### Unreal

Nanite (sem equivalente direto) e Lumen ativados por padrão em projetos novos

### Unity

LODs manuais e contagem de polígonos cuidadosa; GI dinâmico (URP/HDRP) ou bake via Progressive Lightmapper



## **Demonstração: ativando Nanite e Lumen**

---

O professor confirma Lumen como método de Global Illumination e Reflections em Project Settings, habilita Nanite em Static Meshes do nível, e move uma luz em Play para mostrar a resposta em tempo real.

**Resultado esperado:** iluminação indireta muda instantaneamente, sem qualquer bake.

## Tendências de Motores de Jogos

### *Imagem sugerida*

*Objetivo: mostrar o nível de teste antes e depois da ativação de Lumen.*

*Enquadramento: duas capturas lado a lado do mesmo trecho do **Map\_Exploration**.*

*Elementos presentes: à esquerda, iluminação estática/plana; à direita, iluminação indireta dinâmica com sombras suaves.*

*Destaque visual: seta indicando a mudança de intensidade da luz refletida entre as duas capturas.*

*Legenda sugerida: "A mesma cena, antes e depois da iluminação global dinâmica do Lumen."*

Tendências de Motores de Jogos



# O que falta para um jogo rodar fora do Unreal Editor?

Pense em tudo que já existe — BP\_Player, material, Landscape, Nanite, Lumen — e onde isso "mora" hoje.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

## Packaging: do projeto ao executável

- Nenhuma engine entrega um jogo jogável fora do Editor sem conversão explícita
- Compila o projeto para uma plataforma-alvo (neste curso, Windows)
- Empacota todos os assets necessários em um executável autocontido



O momento em que o trabalho do módulo deixa de ser protótipo e vira produto distribuível.

## Packaging: Unreal × Unity

### Unreal

Platforms > Package Project, plataforma-alvo + configuração (Development/Shipping)

### Unity

File > Build Settings, mesmo problema de gerar executável a partir do projeto editável

## **Demonstração: gerando o primeiro build**

O professor confirma `Map_Exploration` como Editor Startup Map e Game Default Map, seleciona Windows + Development em Package Project, e executa o `.exe` gerado fora do Editor.

**Resultado esperado:** BP\_Player controlável por Enhanced Input dentro do executável.

## Arquitetura: do Editor ao Build

- Nível de teste consolidado (material + Landscape) recebe Nanite/Lumen
- Editor Startup Map e Game Default Map apontam para Map\_Exploration
- Packaging gera o executável em Builds/Semana03 (fora da pasta do projeto)

### NA INDÚSTRIA

O mesmo pipeline — configurar mapa padrão, empacotar, testar fora do editor — se repete em qualquer entrega futura da disciplina.

## Boas práticas

### ✓ BOAS PRÁTICAS

- Testar o passo a passo de Packaging na mesma versão de engine antes da aula
- Iniciar o packaging o mais cedo possível no laboratório, nunca no último minuto
- Ajustar iluminação observando o resultado em tempo real do Lumen, não "no escuro"



## Laboratório do Encontro 2

Cada grupo ativa Nanite/Lumen no próprio nível e conduz o packaging do seu projeto até gerar um executável funcional.

### OBJETIVOS

Critério de sucesso: build executável do Vertical Slice, gerado fora do Editor, com Nanite e Lumen ativos e o BP\_Player controlável dentro do executável.

# Checkpoint + Showcase

Cada grupo executa seu build empacotado e demonstra o BP\_Player controlável dentro dele.

## ATENÇÃO

Preenchimento do Modelo de Avaliação de Checkpoint (Rubrica 3) durante o Showcase.

## Resumo da Semana 3

---

- Nanite resolve complexidade geométrica; Lumen resolve iluminação indireta
- Packaging transforma o projeto editável em produto executável
- Primeiro build do Vertical Slice gerado, com BP\_Player funcional dentro dele
- Encerramento do Módulo 1 — Aprender a Ferramenta

## Próximos passos



A Semana 4 abre a Unidade II — Construir Sistemas. GameMode, GameState, PlayerController e GamelInstance passam a organizar, por trás do que já é visível, um jogo que até aqui era apenas um protótipo navegável.

**Leitura recomendada:** Unreal Engine Materials · Nanite Virtualized Geometry · Lumen Global Illumination and Reflections · Packaging Your Project (Epic Games Documentation).